

01 Vectors

1.7 Projection

Scalar projection gives a coordinate, vector projection gives a direction, the orthogonal complement is what's left unexplained



Linear Algebra Made Easy and Visual | Open-source code & drafts: <https://github.com/Visualize-ML>

What You Will Learn

1

Master the definition of scalar projection: the scalar projection of $\mathbf{a}$ onto $\mathbf{b} = \|\mathbf{a}\|\cos\theta = \mathbf{a}\cdot\mathbf{b}/\|\mathbf{b}\|$, a single scalar "coordinate value"

3

Understand the orthogonal complement: the remaining part of a vector that cannot be expressed along the projection direction, perpendicular to it

5

Understand the basic concepts of standard/orthonormal basis, basis, basis vector, linear independence, and linear combination

2

Master the definition of vector projection: vector projection = scalar projection $\times$ direction vector, giving both "coordinate + direction" at once

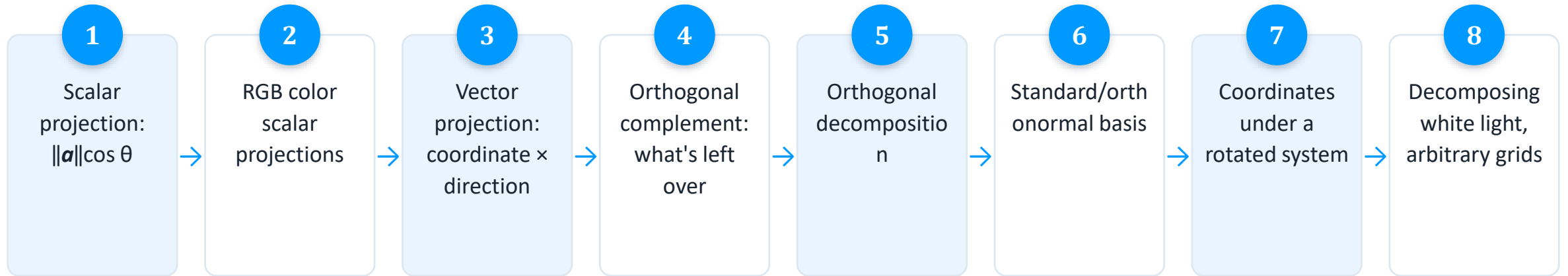
4

Master orthogonal decomposition: splitting a vector into a projection and an orthogonal complement, two mutually perpendicular components

6

Use scalar projection to compute a vector's coordinates under any "rotated square grid" (rotated rectangular coordinate system)

How the Ideas Connect



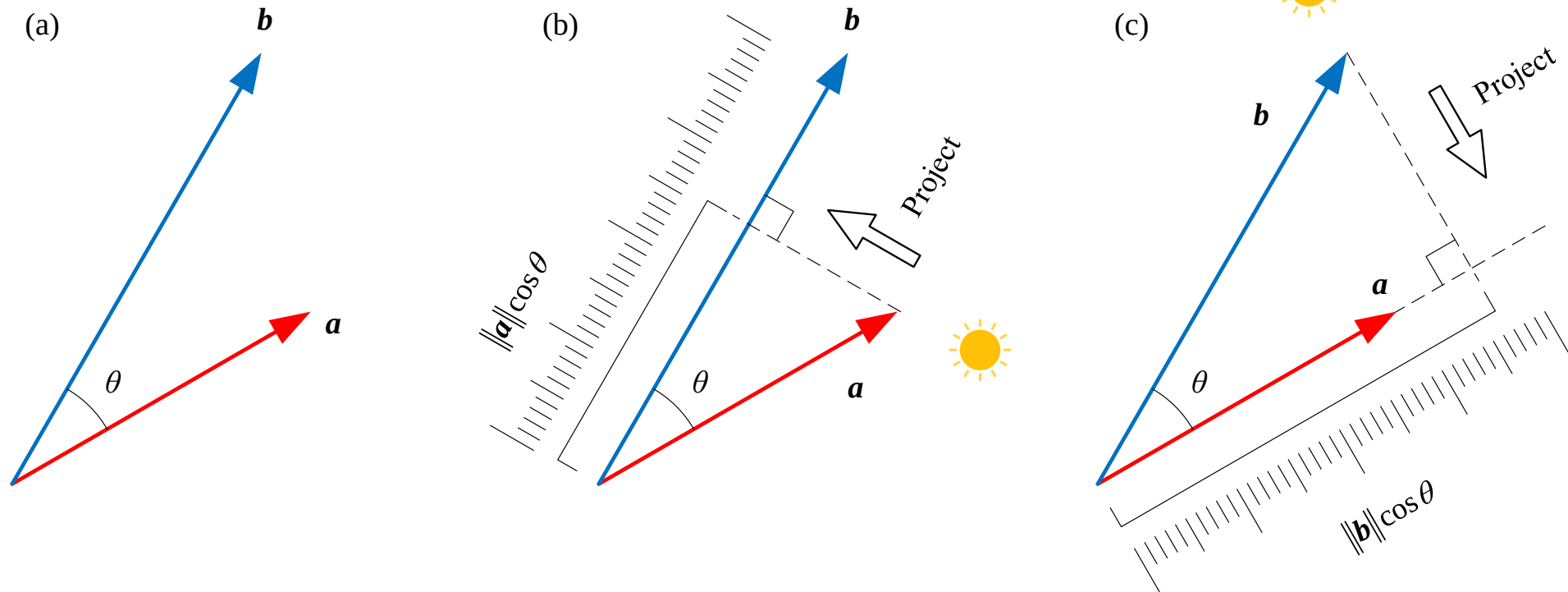
Scalar Projection: A Vector's "Coordinate" Along a Direction

$$\text{Scalar projection of } \mathbf{a} \text{ onto } \mathbf{b} = \|\mathbf{a}\| \cos \theta = \mathbf{a} \cdot \mathbf{b} / \|\mathbf{b}\|$$

- For nonzero vectors $\mathbf{a}, \mathbf{b}$ with angle θ between them, the scalar projection is a single number carrying no direction information
- Think of it as the "coordinate value" where $\mathbf{a}$ lands after orthogonal projection onto $\mathbf{b}$ — the length of $\mathbf{a}$'s "shadow" along $\mathbf{b}$
- Symmetrically, the scalar projection of $\mathbf{b}$ onto $\mathbf{a}$ is $\|\mathbf{b}\| \cos \theta = \mathbf{a} \cdot \mathbf{b} / \|\mathbf{a}\|$ — the same relationship, reversed

Figure 1 · Scalar Projection in Two Directions

- (a) θ is the angle between vectors $\mathbf{a}$ and $\mathbf{b}$
- (b) $\|\mathbf{a}\|\cos\theta$ is the scalar projection of $\mathbf{a}$ onto $\mathbf{b}$; the dashed line is perpendicular to $\mathbf{b}$, like measuring a shadow's length in sunlight
- (c) $\|\mathbf{b}\|\cos\theta$ is the scalar projection of $\mathbf{b}$ onto $\mathbf{a}$; the dashed line is perpendicular to $\mathbf{a}$



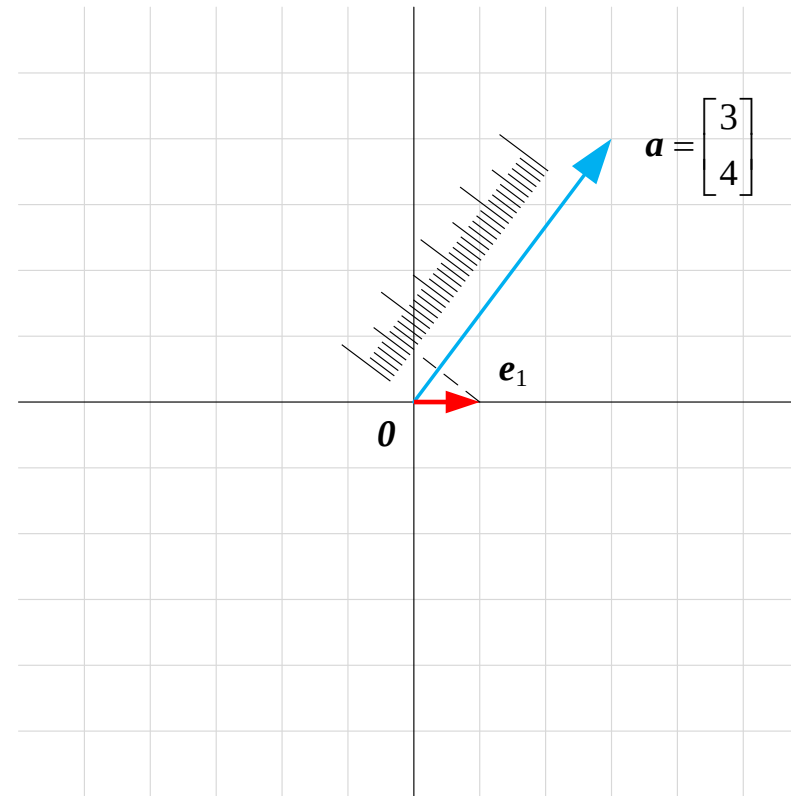
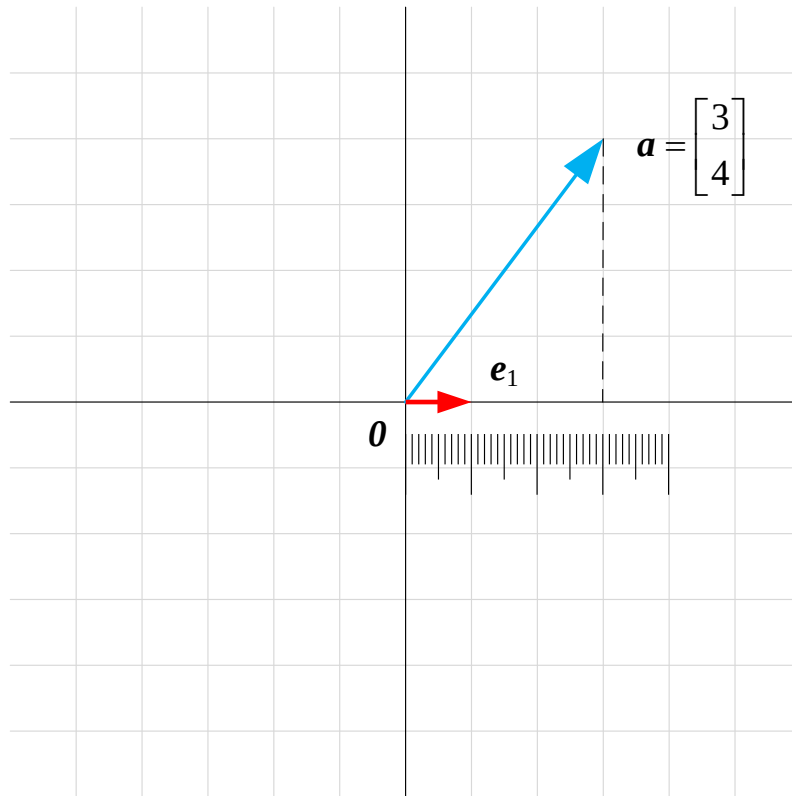
A Scalar Projection Calculation

$$\mathbf{a}=[3,4]^T, \text{ scalar projection onto } \mathbf{e}_1 = \mathbf{a} \cdot \mathbf{e}_1 = 3$$

- $\mathbf{e}_1=[1,0]^T$ is itself a unit vector, so the inner product alone gives the scalar projection — no division by length is needed
- In reverse, the scalar projection of $\mathbf{e}_1$ onto $\mathbf{a}$ is $\mathbf{e}_1 \cdot (\text{the direction vector of } \mathbf{a}) = 3/5$
- $\mathbf{a}$ has length 5, so $\mathbf{e}_1$'s "shadow" along $\mathbf{a}$ covers 3/5 of $\mathbf{a}$'s length

Figure 2 · Mutual Scalar Projections of $\mathbf{a}$ and $\mathbf{e}_1$

- Left: the scalar projection of $\mathbf{a}=[3,4]^T$ onto $\mathbf{e}_1$ is 3 — the coordinate value where the dashed line meets the x_1 axis
- Right: the scalar projection of $\mathbf{e}_1$ onto $\mathbf{a}$ is $3/5$; the slanted ruler in the figure marks the projection direction



Computing Scalar Projection With NumPy

Inner product divided by length gives the scalar projection

- Define vectors $a=[3,4]$, $e1=[1,0]$ with `np.array()`
- `np.dot(a, e1)` and `np.dot(e1, a)` return the same value, confirming the commutativity of the inner product
- `np.linalg.norm()` computes the lengths of a and $e1$ separately
- Scalar projection = inner product / length of the target direction vector, giving the projections of a onto $e1$ and of $e1$ onto a

```
## 初始化
import numpy as np

## 定义向量
a = np.array([3, 4])
e1 = np.array([1, 0])

## 计算点积
dot_a_e1 = np.dot(a, e1)
dot_e1_a = np.dot(e1, a)

## 计算向量模长
norm_a = np.linalg.norm(a)
norm_e1 = np.linalg.norm(e1)

## 计算标量投影
scalar_proj_a_on_e1 = dot_a_e1 / norm_e1 # a 在 e1 上的投影
scalar_proj_e1_on_a = dot_e1_a / norm_a # e1 在 a 上的投影
```



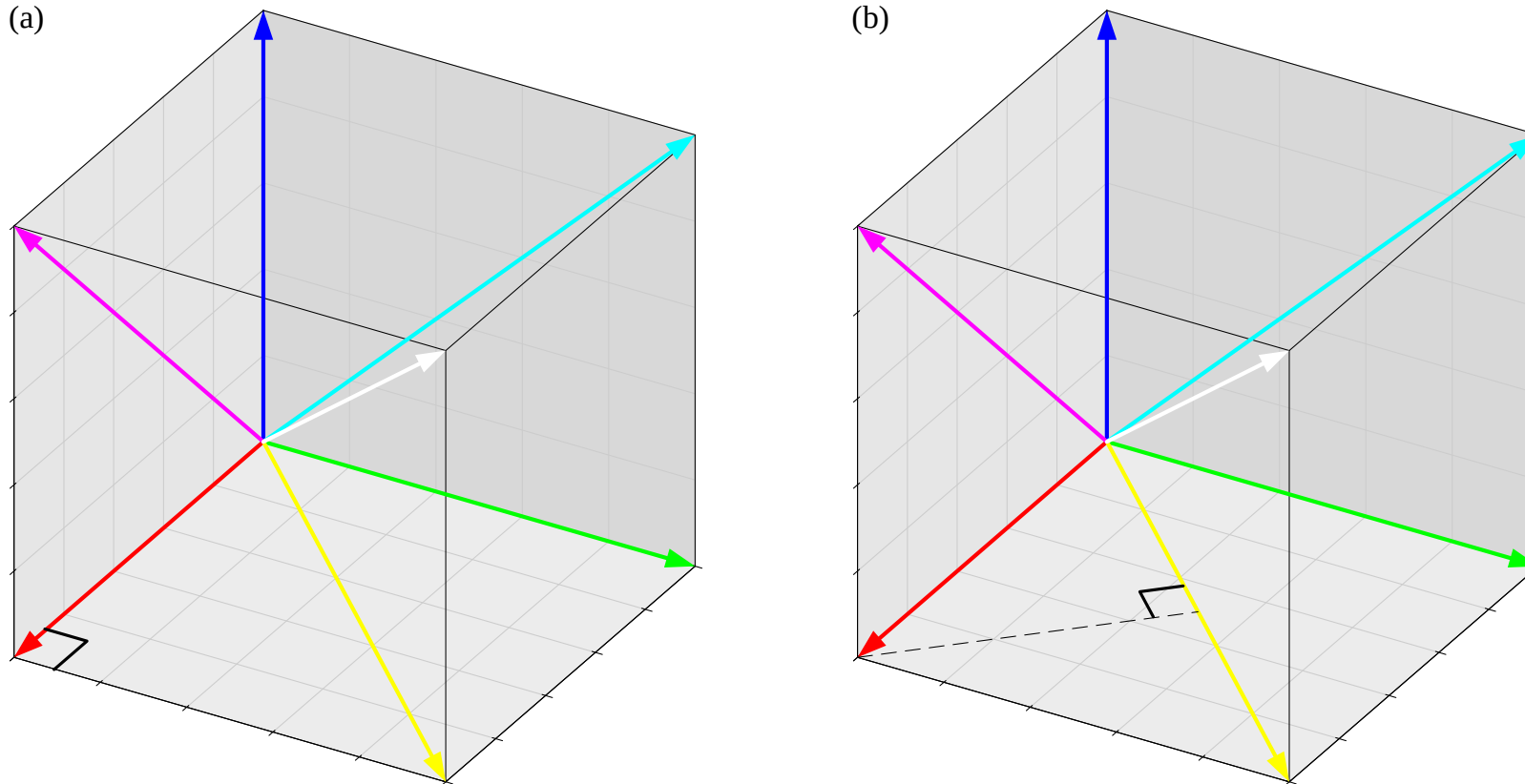
The Scalar Projection of Yellow Onto Red

$$[1,1,0]^T \cdot [1,0,0]^T / \|[1,0,0]^T\| = 1$$

- Yellow light passed through a red filter keeps only its red intensity — that intensity is exactly the scalar projection of yellow onto the red axis
- The scalar projection of yellow $[1,1,0]^T$ onto red $[1,0,0]^T$ is 1, meaning yellow contains 100% of red's intensity
- In reverse, the scalar projection of red onto yellow is $\sqrt{2}/2$; yellow has length $\sqrt{2}$, and red accounts for half of it

Figure 3 · Mutual Scalar Projections of Yellow and Red

- (a)(b) Two viewpoints showing the relationships among red, green, blue, cyan, magenta, yellow, and white in RGB space
- As an exercise, compute the mutual scalar projections of magenta and blue



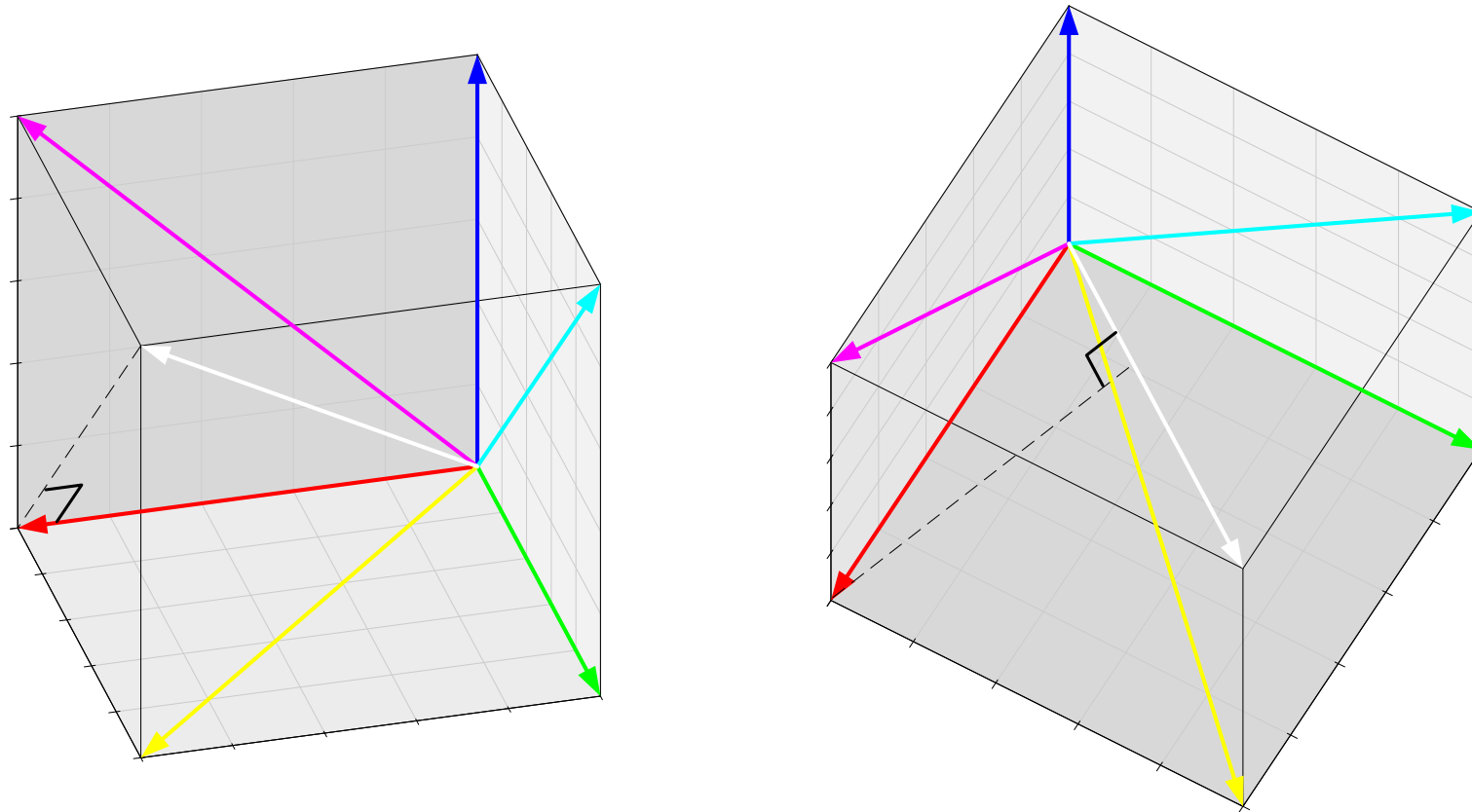
Cyan's Scalar Projection Onto Red Is Zero

$$[0,1,1]^T \cdot [1,0,0]^T / \|[1,0,0]^T\| = 0$$

- Cyan light carries no red component, so its scalar projection onto red is 0
- More broadly, every vector in the blue-green plane has a scalar projection of 0 onto red
- White $[1,1,1]^T$ has a scalar projection of 1 onto red — white contains 100% of red's intensity; in reverse, red's scalar projection onto white is $\sqrt{3}/3$

Figure 4 · Mutual Scalar Projections of White and Red (Two Views)

- Two different viewing angles are used to make the projection relationship clearer
- As an exercise, compute the mutual scalar projections of white and blue, and of white and green



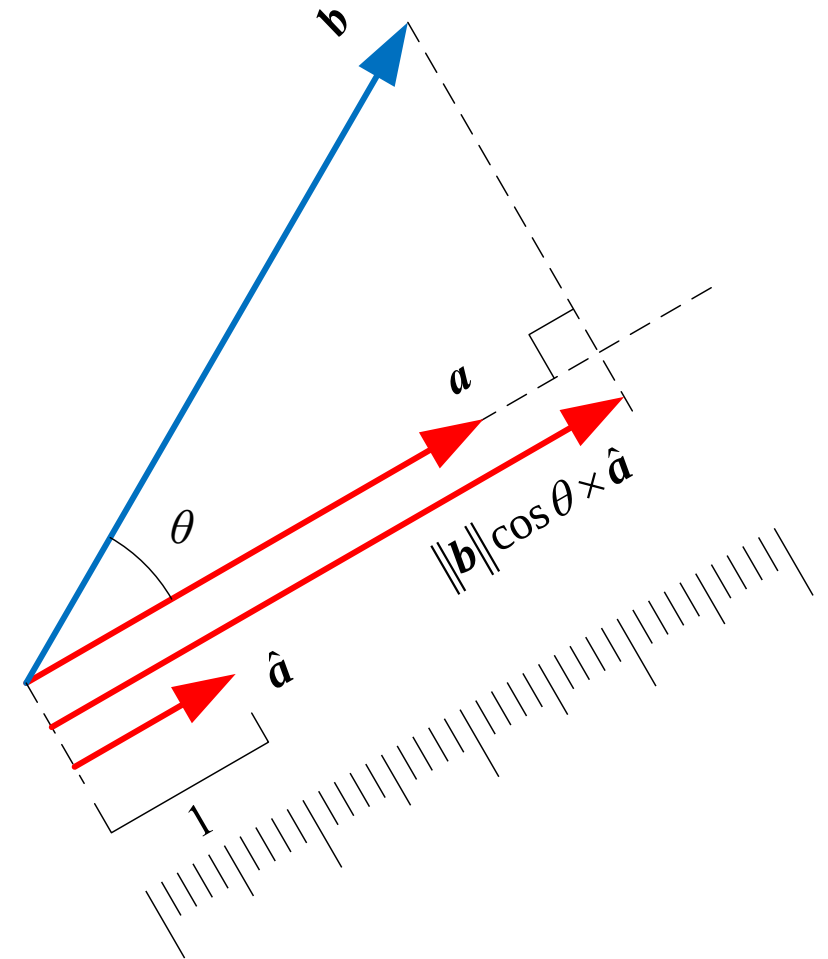
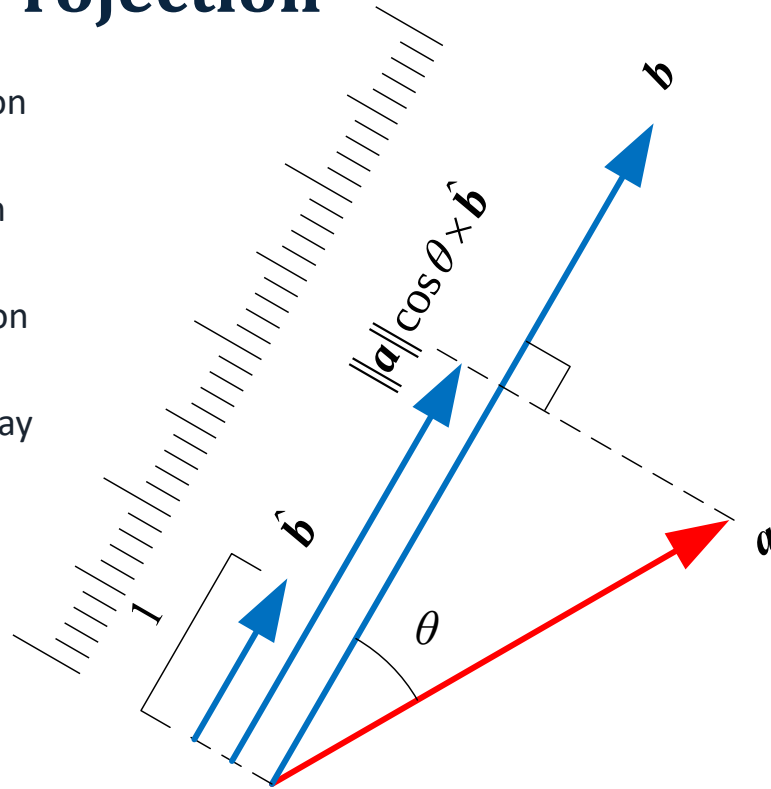
Vector Projection: Scalar Projection × Direction Vector

$$\text{proj}_b a = (a \cdot b / \|b\|^2) b$$

- Scalar projection gives only the "coordinate" along a direction; vector projection adds the direction vector, specifying "coordinate + direction" together
- $\text{proj}_b a$ is parallel to b — it is the part of a that can be expressed as a linear multiple of b
- In reverse, the vector projection of b onto a is $\text{proj}_a b = (a \cdot b / \|a\|^2) a$

Figure 5 · Vector Projection

- (a) $\text{proj}_b a$ is the vector projection of a onto b ; its length equals the scalar projection, and its direction matches b
- (b) $\text{proj}_a b$ is the vector projection of b onto a , with length and direction determined the same way



Mutual Vector Projections of Yellow and Red

$$\text{Vector projection of red onto yellow} = (\sqrt{2}/2 \times 1/\sqrt{2}) \times [1,1,0]^T = [1/2, 1/2, 0]^T$$

- Red $[1,0,0]^T$ is itself a unit vector, so the vector projection of yellow $[1,1,0]^T$ onto red is $1 \times [1,0,0]^T = [1,0,0]^T$
- Yellow's unit vector is $(1/\sqrt{2}) \times [1,1,0]^T$; combined with the scalar projection $\sqrt{2}/2$ found earlier, red's vector projection onto yellow is $[1/2, 1/2, 0]^T$
- Vector projection keeps the direction information, so the result is a concrete vector, not just a number

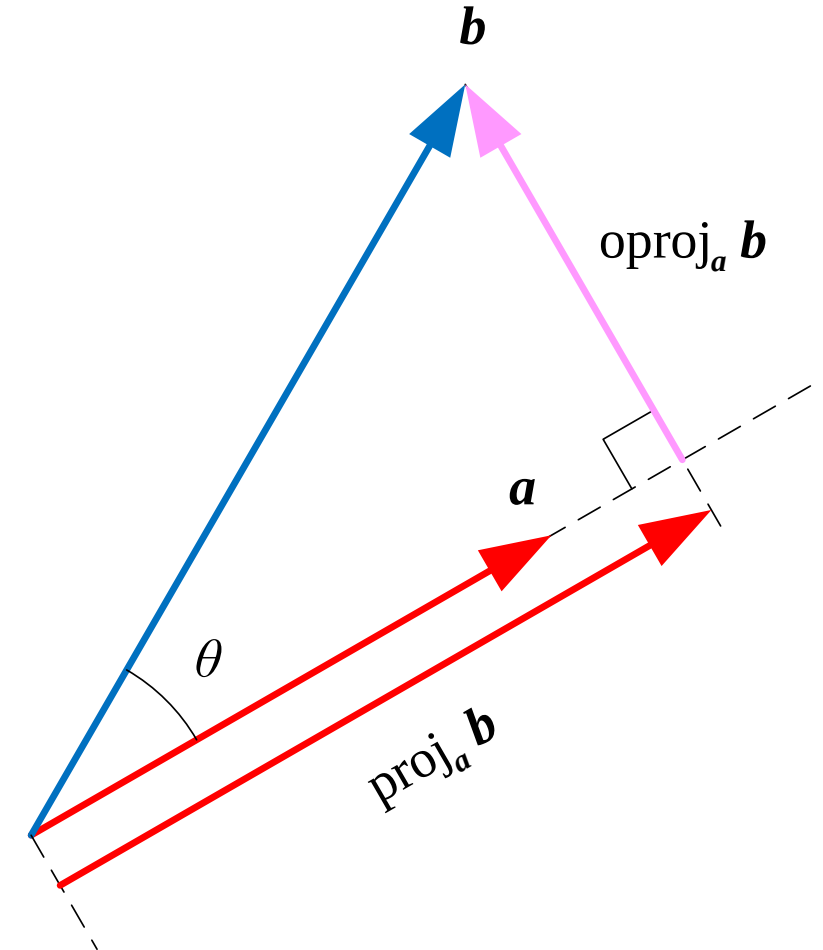
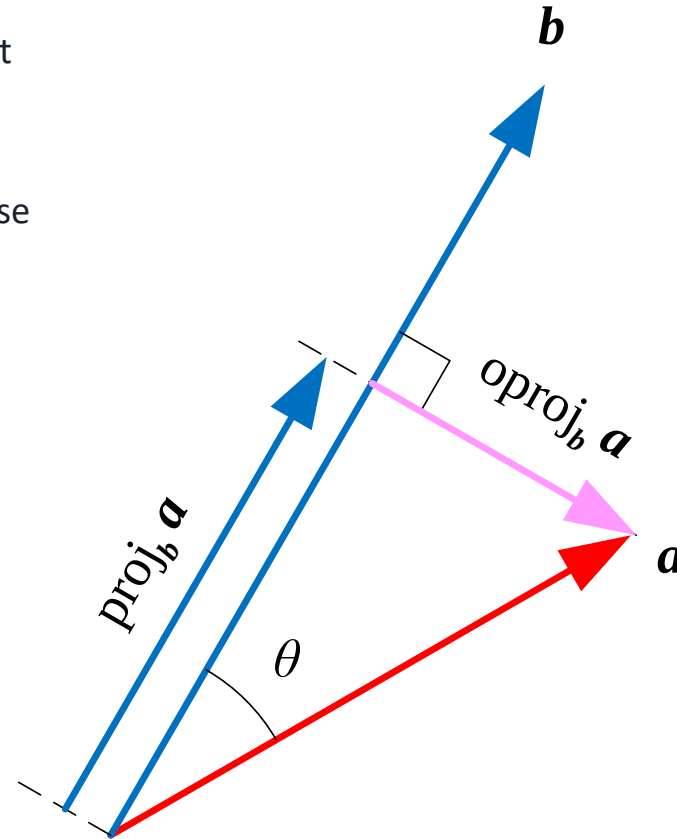
Orthogonal Complement: What's Left, Perpendicular to the Projection

$$\text{oproj}_b a = a - \text{proj}_b a$$

- After a vector is orthogonally projected onto a direction, whatever remains that cannot be expressed along that direction is its orthogonal complement
- Also called the orthogonal remainder, the perpendicular component, or the normal component
- $\text{oproj}_b a$ is the part of a that cannot be linearly expressed by b , and it is always perpendicular to b

Figure 6 · Orthogonal Complement

- Left: $\text{oproj}_b a$ (pink arrow) is the part of a that b cannot express, perpendicular to $\text{proj}_b a$
- Right: $\text{oproj}_a b$ (pink arrow) is likewise the part of b that a cannot express, perpendicular to a



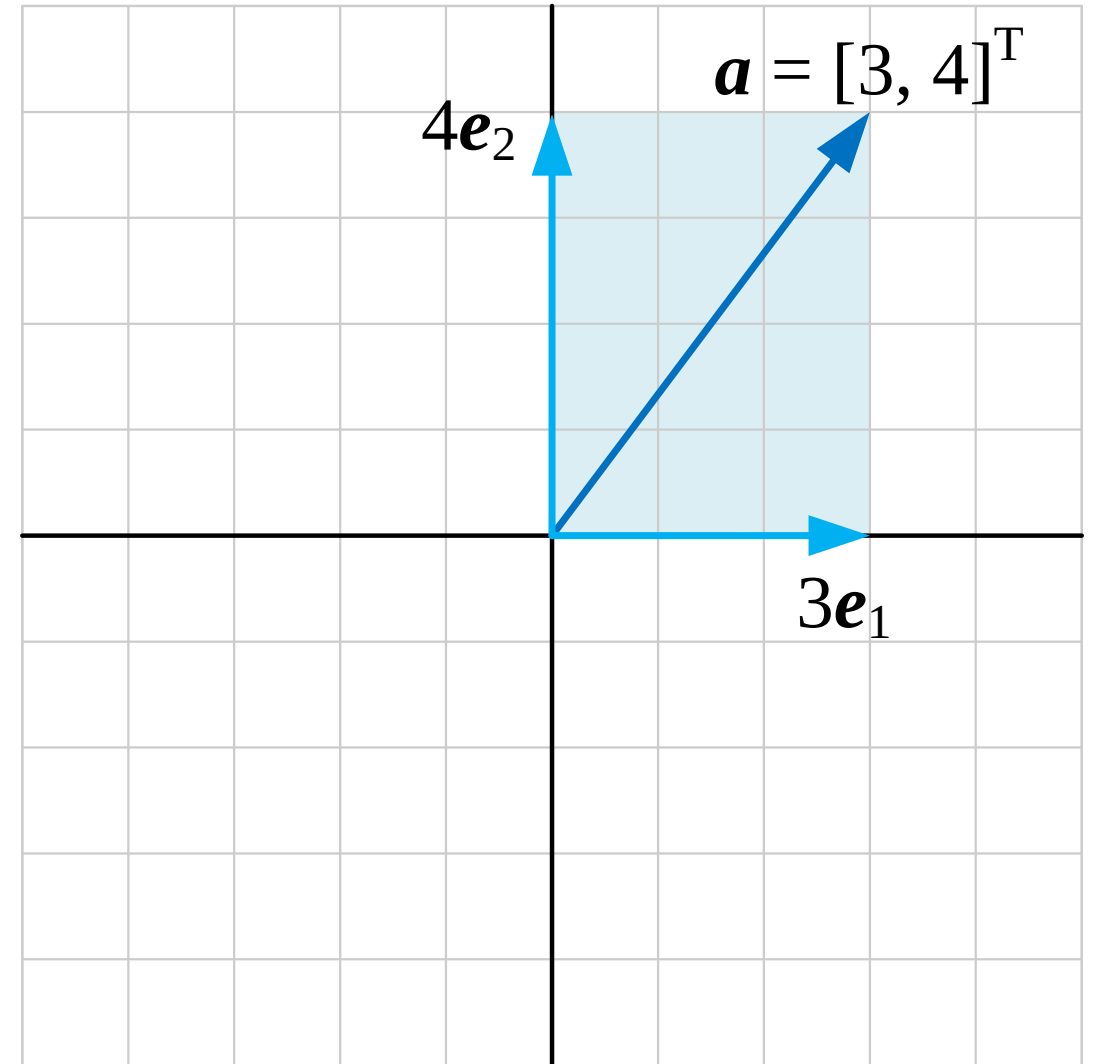
Splitting a Vector Into Two Axis-Aligned Components

$$\mathbf{x} = x_1 \mathbf{e}_1 + x_2 \mathbf{e}_2; \text{ example } \mathbf{a} = [3, 4]^T = 3\mathbf{e}_1 + 4\mathbf{e}_2$$

- The core idea of vector decomposition: split a vector into two or more components that sum back to the original vector
- In the plane, every vector can be written as the sum of two components along the coordinate axes
- For $\mathbf{a} = [3, 4]^T$, the horizontal component is $[3, 0]^T = 3\mathbf{e}_1$ and the vertical component is $[0, 4]^T = 4\mathbf{e}_2$

Figure 7 · Decomposing $\mathbf{a}$ in Terms of $[\mathbf{e}_1, \mathbf{e}_2]$

- The horizontal component $3\mathbf{e}_1$ and vertical component $4\mathbf{e}_2$ of $\mathbf{a}=[3,4]^T$ are perpendicular, and together they sum to $\mathbf{a}$
- This is exactly orthogonal decomposition: splitting a vector into components that do not interfere with each other



Basis, Basis Vectors, and the Standard Basis

Basis A set of linearly independent vectors in a vector space whose linear combinations can express any vector in that space

Basis Vector Each vector in a basis — like a beam supporting the grid structure of the space

Linearly Independent No vector in the set can be written as a linear combination of the others — each is independent in direction

Linear Combination The operation of scaling a set of vectors and adding them together to form a new vector

Standard / Natural The unit vectors along the positive coordinate axes are usually chosen as basis vectors — the most natural, most common basis

Standard Basis $[\mathbf{e}_1, \mathbf{e}_2]$ The basis formed by $\mathbf{e}_1$, $\mathbf{e}_2$, usually representing the positive directions of the Euclidean coordinate axes

Any Vector Splits Into a Projection Plus an Orthogonal Complement

$$\mathbf{a} = \text{proj}_{\mathbf{b}} \mathbf{a} + \text{oproj}_{\mathbf{b}} \mathbf{a}; \mathbf{b} = \text{proj}_{\mathbf{a}} \mathbf{b} + \text{oproj}_{\mathbf{a}} \mathbf{b}$$

- $\text{proj}_{\mathbf{b}} \mathbf{a}$ and $\text{oproj}_{\mathbf{b}} \mathbf{a}$ are mutually perpendicular — the first is parallel to $\mathbf{b}$, the second perpendicular to $\mathbf{b}$
- Symmetrically, $\mathbf{b}$ also splits into a vector projection parallel to $\mathbf{a}$ plus an orthogonal complement perpendicular to $\mathbf{a}$
- This decomposition relies on the inner product's definition in Euclidean space; its existence and uniqueness are guaranteed, with broad applications

Figure 8 · Orthogonal Vector Decomposition

- (a) $\mathbf{a}$ splits into $\text{proj}_{\mathbf{b}} \mathbf{a}$ (parallel to $\mathbf{b}$) plus $\text{oproj}_{\mathbf{b}} \mathbf{a}$ (perpendicular to $\mathbf{b}$)
- (b) $\mathbf{b}$ splits into $\text{proj}_{\mathbf{a}} \mathbf{b}$ (parallel to $\mathbf{a}$) plus $\text{oproj}_{\mathbf{a}} \mathbf{b}$ (perpendicular to $\mathbf{a}$)

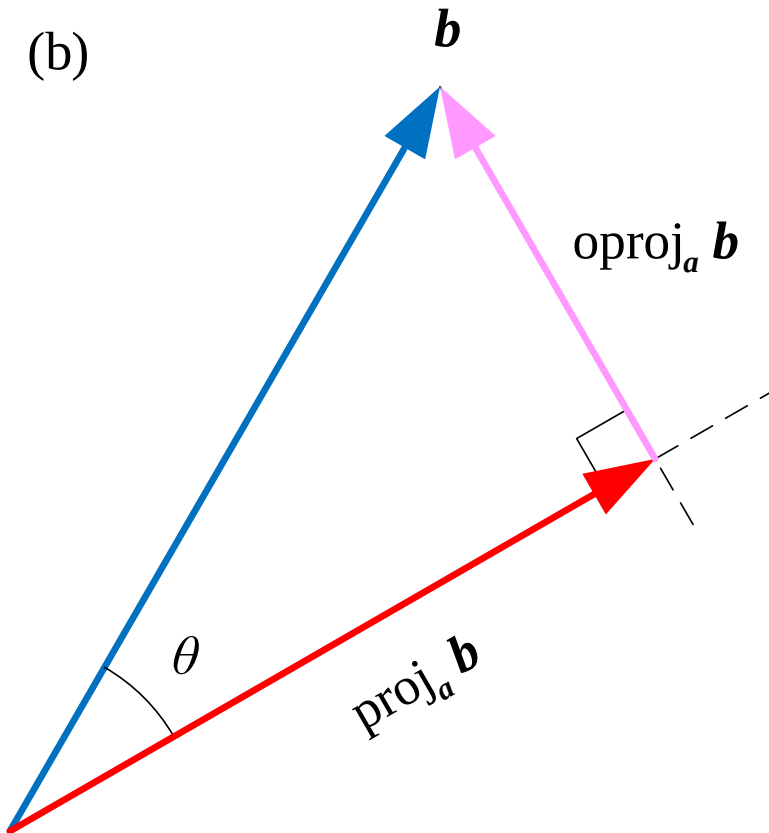
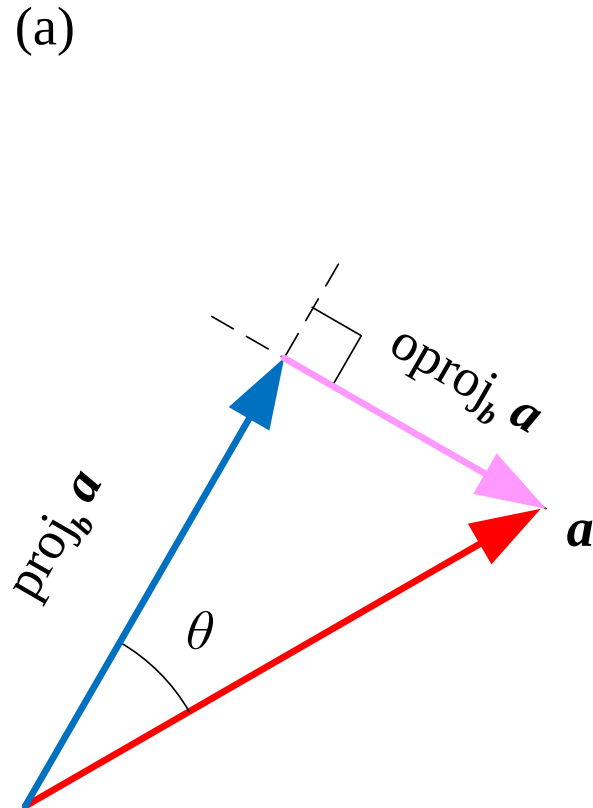
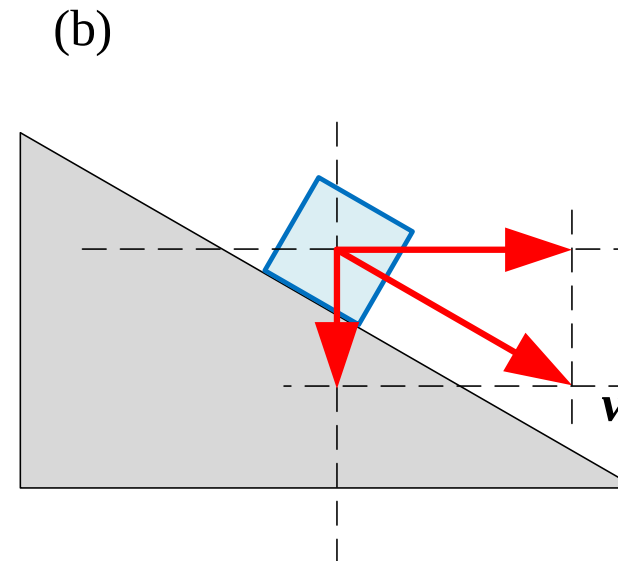
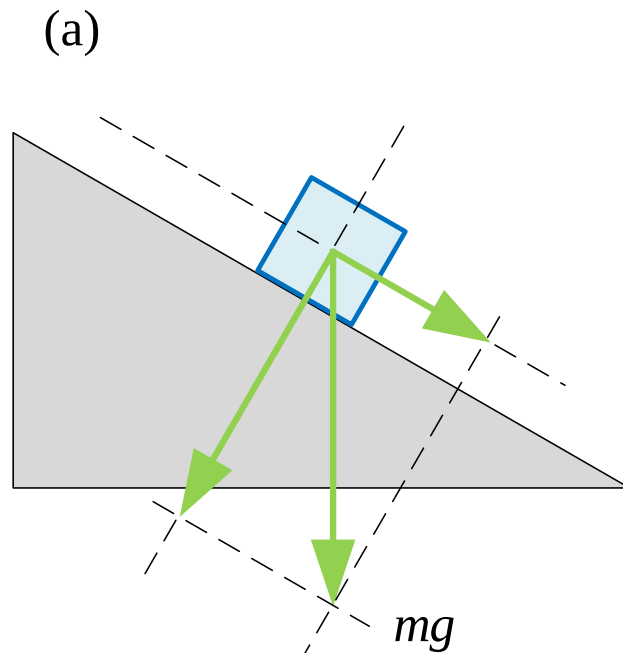


Figure 9 · Orthogonal Decomposition of Force and Velocity

- (a) Gravity acting on an object on a slope splits into a component down the slope and a component perpendicular to the slope
- (b) The velocity of an object moving down a slope splits into horizontal and vertical components — a familiar example from introductory physics



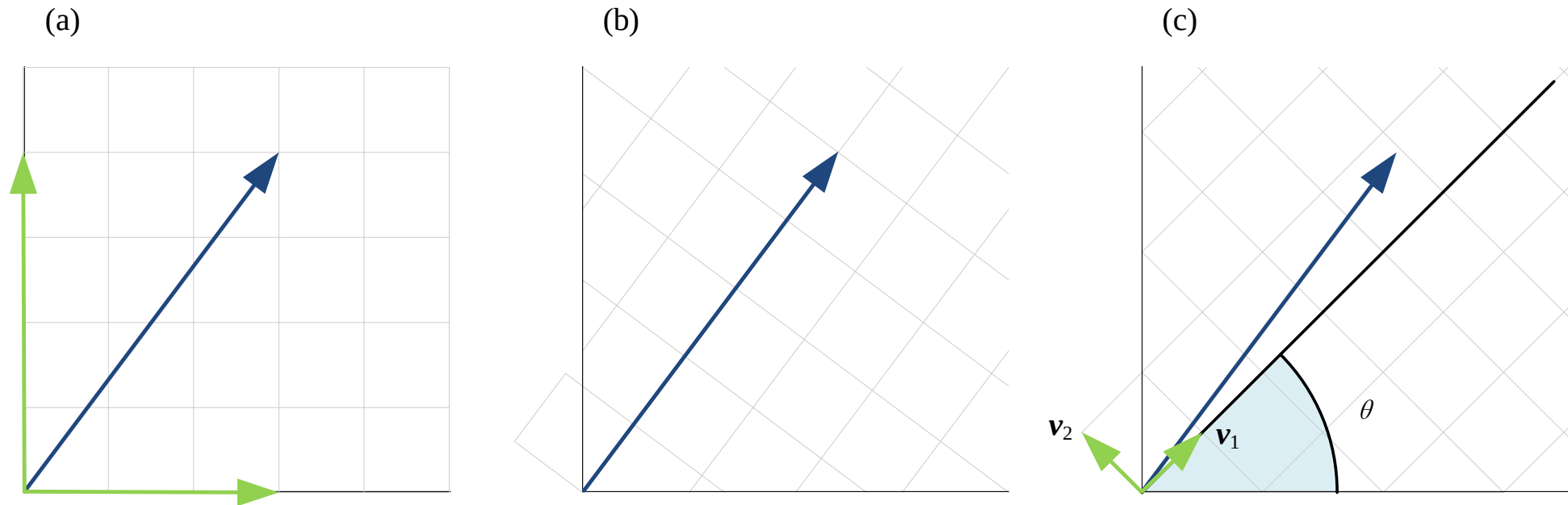
A Rotated Square Grid: Any Direction Can Be a "Right-Angle" System

$$\mathbf{v}_1 = [\cos \theta; \sin \theta], \mathbf{v}_2 = [-\sin \theta; \cos \theta]$$

- The standard rectangular coordinate system is just one of infinitely many "rotated square grids"; $\mathbf{v}_1, \mathbf{v}_2$ form an orthonormal set
- For any θ , $\mathbf{v}_1$ and $\mathbf{v}_2$ both have length 1 and their inner product is 0 — together they form a new "right-angle" coordinate system
- The coordinates of $\mathbf{a}=[4,3]^T$ in this new grid can each be found via scalar projection

Figure 10 · Orthogonal Decomposition in a Rotated Coordinate System

- (a) In the familiar standard coordinate system, $\mathbf{a}=[3,4]^T$ has coordinates (3, 4)
- (c) In the "rotated square grid" with axes $\mathbf{v}_1, \mathbf{v}_2$, $\mathbf{a}$'s coordinates become (5, 0)



Computing $\mathbf{a}$'s New Coordinates in the Rotated System

$$\mathbf{a} \cdot \mathbf{v}_1 = 3\cos \theta + 4\sin \theta$$

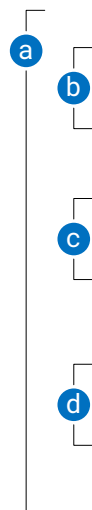
$$\mathbf{a} \cdot \mathbf{v}_2 = 3\sin \theta - 4\cos \theta$$

- These two values are the scalar projections of $\mathbf{a}$ onto $\mathbf{v}_1$ and $\mathbf{v}_2$ — exactly its coordinates in the new system
- Adding the vector projections along both directions recovers the original vector $\mathbf{a} = [3, 4]^T$, confirming the decomposition
- As θ varies, infinitely many different "rotated square grids" appear, each a valid "right-angle" coordinate system for describing $\mathbf{a}$

Implementing Orthogonal Vector Decomposition

The `decompose()` function returns the projection components along both directions

- Define a function `decompose(a, theta)` that takes a vector `a` and an angle `theta` (radians), returning the components along two orthogonal directions
- Use `theta` to build two mutually perpendicular unit vectors `v1`, `v2`, computed with `np.cos` and `np.sin`
- Use `np.dot()` to compute the scalar projections of `a` onto `v1` and `v2`, then multiply by the corresponding unit vectors to get the vector projections
- Verify: the two vector projections, added together, equal the original vector `a`



```
## 初始化
import numpy as np

## 定义向量 a
a = np.array([3,4])

## 自定义函数
def decompose(a, theta):
    v1 = np.array([np.cos(theta), np.sin(theta)])
    v2 = np.array([-np.sin(theta), np.cos(theta)])

    # 标量投影 (坐标)
    a_v1 = np.dot(a, v1)
    a_v2 = np.dot(a, v2)

    # 向量投影 (正交分解)
    proj_a_v1 = a_v1 * v1
    proj_a_v2 = a_v2 * v2

    return proj_a_v1, proj_a_v2

## 正交分解
theta = 30
theta = theta / 180 * np.pi

proj_a_v1, proj_a_v2 = decompose(a, theta)

## 验证
proj_a_v1 + proj_a_v2
```



Multiple Orthogonal Decompositions of White in RGB Space

$$\text{White } \mathbf{w} = \text{red} + \text{green} + \text{blue} = \mathbf{e}_1 + \mathbf{e}_2 + \mathbf{e}_3$$

- White can also decompose orthogonally as yellow + blue, magenta + green, or cyan + red — three more pairwise-orthogonal decompositions
- Red, green, and blue are themselves mutually orthogonal, so "red + green + blue" is itself one valid orthogonal decomposition
- Vector decomposition is never unique — the same vector admits many different orthogonal decompositions

Figure 11 · Three Orthogonal Decompositions of White

- (a) White decomposes into yellow and blue: blue is perpendicular to yellow, and also to the x_1x_2 plane
- (b) White decomposes into magenta and green: green is perpendicular to magenta, and also to the x_1x_3 plane
- (c) White decomposes into cyan and red: red is perpendicular to cyan, and also to the x_2x_3 plane

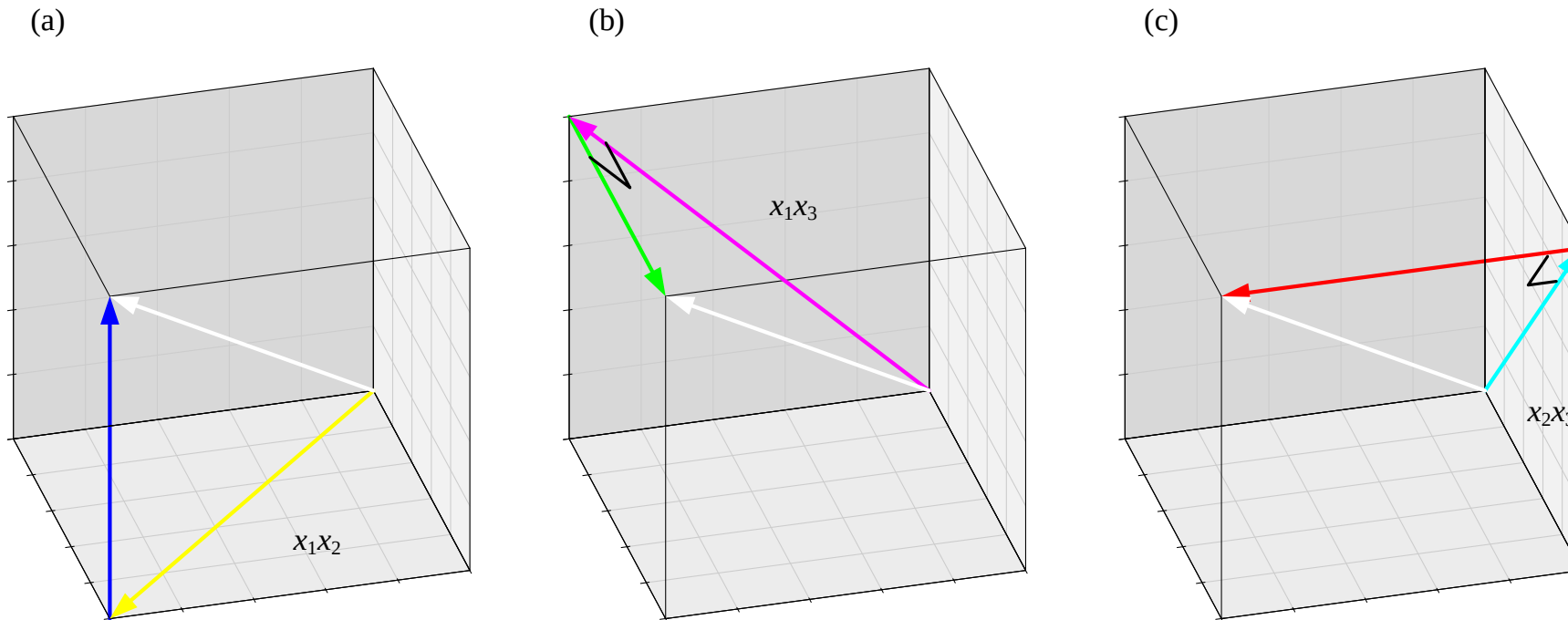


Figure 12 · Different Decompositions of $a=[3,4]^T$

- Six different grids (bases) decompose the same vector a : a rectangular grid, a rotated square grid, general parallelogram grids, and more
- These grids differ in shape, but all share the same traits — parallel, evenly spaced, and passing through the origin
- There are infinitely many ways to decompose a — and correspondingly infinitely many kinds of grids (bases) that can tile the entire plane

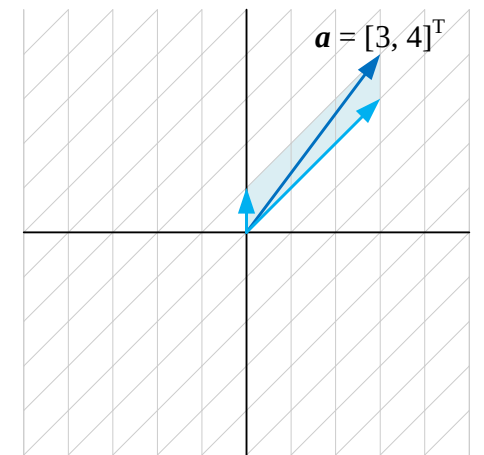
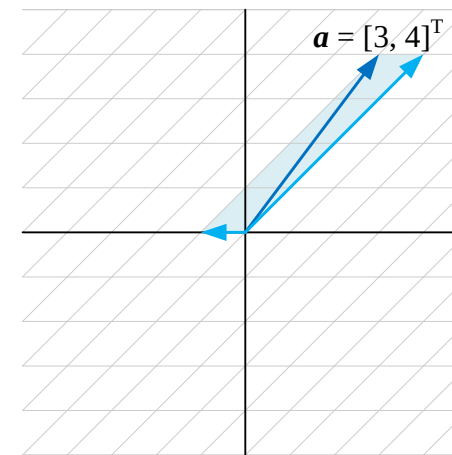
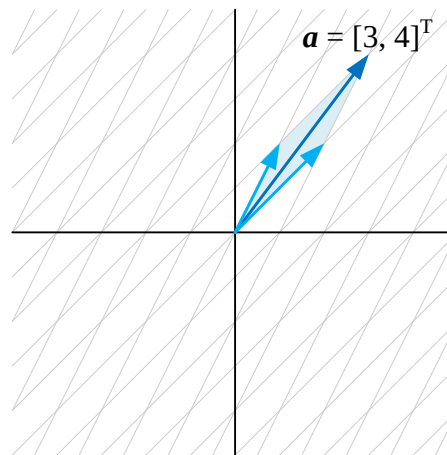
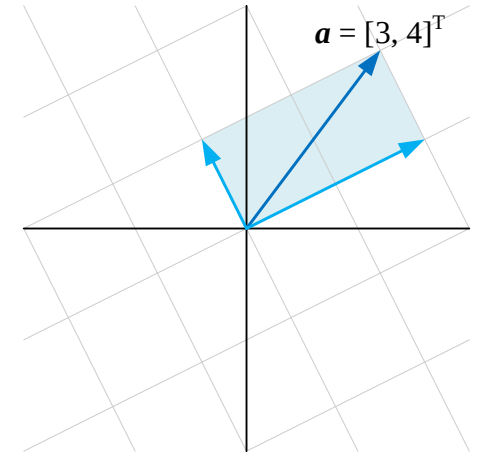
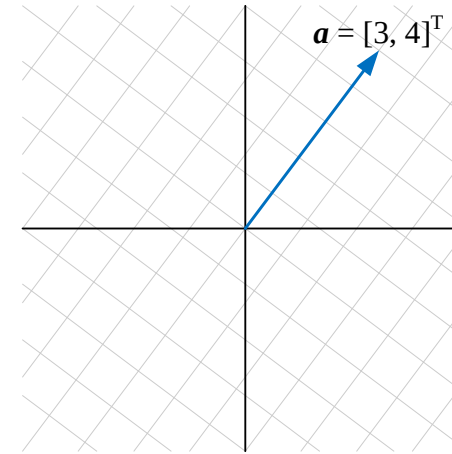
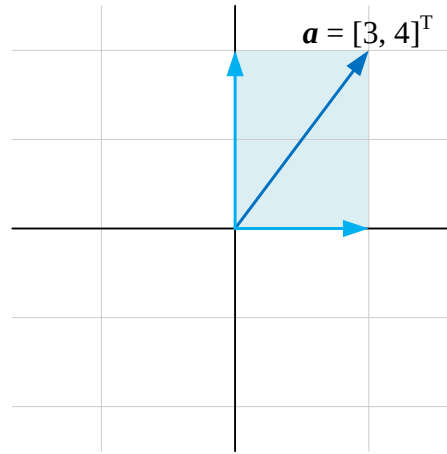
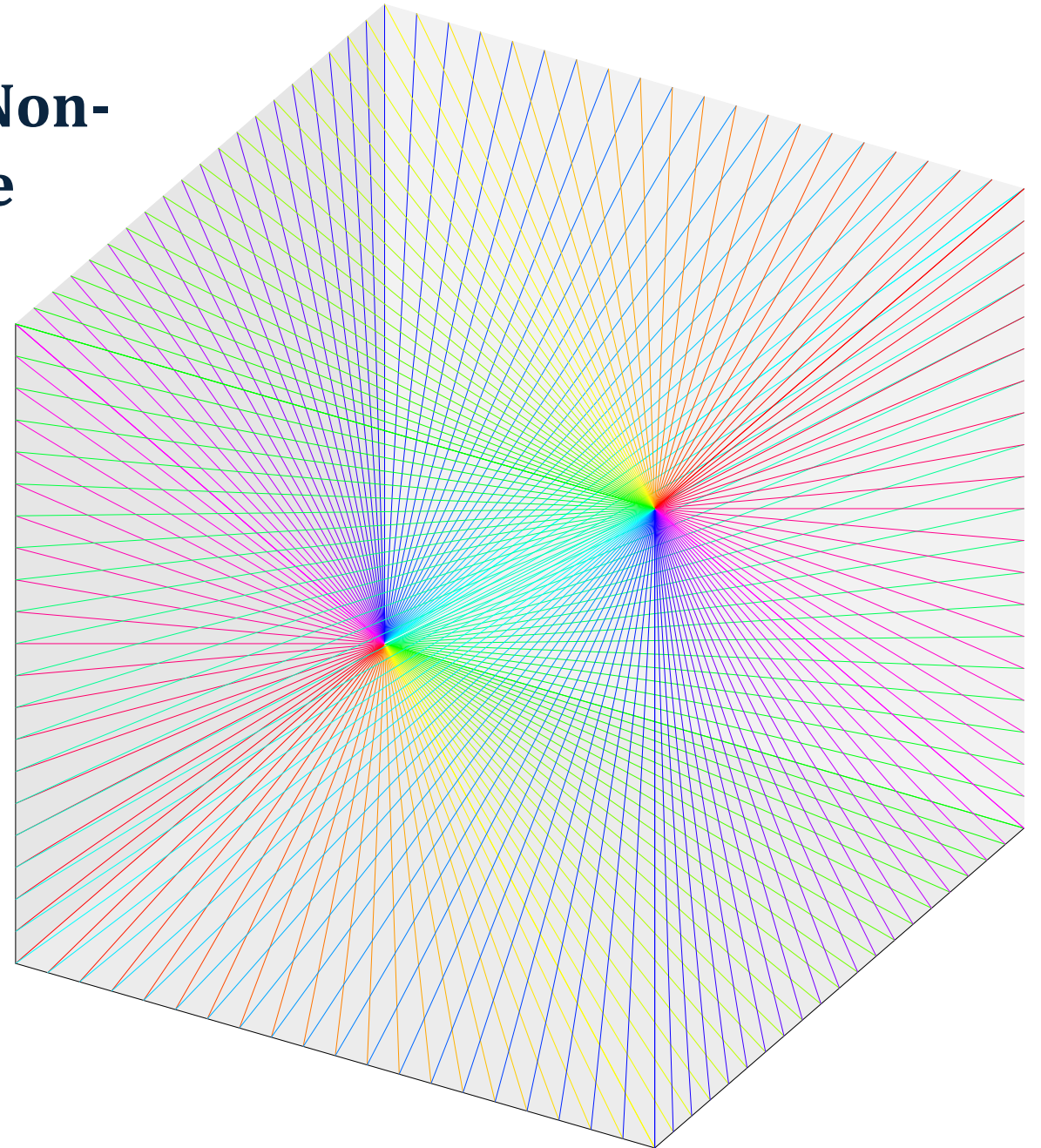


Figure 13 · Further Orthogonal and Non-Orthogonal Decompositions of White

- Generate a random color vector in RGB space, take the difference between it and the white vector, then chain them tip-to-tail to recover the white vector
- Vector decomposition can be seen as the inverse of vector addition — given a sum, work backward to find components that satisfy it
- There are nearly infinite such decompositions; the dense web of colored lines in the figure visualizes just how many are possible



Key Terms in This Section

Scalar Projection $\|a\|\cos\vartheta = a \cdot b / \|b\|$ — the result is a scalar

Orthogonal Complement *Orthogonal Rejection* — the part of a that b cannot linearly express

Basis / Basis Vector A set of linearly independent vectors, and each vector within it

Standard Basis The unit vectors along the positive coordinate axes, e.g. $[e_1, e_2]$

Rotated Coordinate System A new coordinate system formed by a rotated orthonormal pair of unit vectors

Vector Projection Scalar projection $\times$ direction vector, $\text{proj}_b a$

Orthogonal Decomposition Splitting into a projection and an orthogonal complement, two perpendicular parts

Linearly Independent No vector in a set can be written as a linear combination of the others

Orthonormal Vectors A set of vectors that are pairwise orthogonal and each have length 1

Coordinates via Scalar Projection Using inner products of a with new basis vectors to find a 's new coordinates

GO FURTHER

Companion Code and Further Reading

Code 1 · LA_01_07_01.ipynb

Compute scalar projection and verify commutativity

Code 2 · LA_01_07_03.ipynb

Implement orthogonal vector decomposition via `decompose()`

Further Reading · LA_01_06_02.ipynb

Computing vector projection — see the previous section's code

Open-Source Repository

All code and figures for the book: <https://github.com/Visualize-ML>

Q1 – Q8

Q1 Write code to compute the scalar projection of $a=[3,4]^T$ onto $[1,1]$, and verify it by hand.

Q3 Given vectors $a=[3,4]^T$ and $b=[-1,0]$, decompose a into a component parallel to b and a component perpendicular to b .

Q5 Write a function `scalar_projection(a, b)` that computes the scalar projection of a onto b , and check it against a hand calculation.

Q7 Write a function `orthogonal_rejection(a, b)` that computes the orthogonal complement of a with respect to b , and verify its inner product with b is 0.

Q2 Write code to compute the vector projection of $a=[3,4]^T$ onto $[-1,1]$, and verify it by hand.

Q4 Try sketching 10 vector decompositions similar to Figure 12.

Q6 Write a function `vector_projection(a, b)` that returns the vector projection of a onto b (a vector, not a scalar).

Q8 Use Python to verify that for any nonzero vector a , its orthogonal complement plus its vector projection along the projection direction equals a .

Q9 – Q16

Q9 Randomly generate 10 pairs of 2-D vectors, compute their mutual scalar projections, and count how many pairs give a negative scalar projection.

Q11 Use NumPy to verify the mutual scalar projections of magenta and blue in RGB space, and compare with the red-yellow result.

Q13 Use Python to compute the mutual scalar projections of white and blue, and of white and green, and compare with the white-red result.

Q15 Use Matplotlib to reproduce the force-decomposition diagram in Figure 9, and observe how the two components change as the slope angle varies.

Q10 Use Matplotlib to plot a vector a along with its scalar projection, vector projection, and orthogonal complement onto a direction b (as in Figures 1, 5, and 6).

Q12 Write a function `rgb_projection_report(color1, color2)` that reports the scalar projection, vector projection, and angle between two color vectors.

Q14 Write a function `orthogonal_decompose(a, b)` that returns both the vector projection and the orthogonal complement of a with respect to b , and verify they sum to a .

Q16 Given a rotation angle θ , write a function `rotated_basis(theta)` that returns an orthonormal pair v_1, v_2 , and verify each has length 1 with inner product 0.

Q17 – Q24

Q17 Use scalar projection to find the coordinates of $a=[4,3]^T$ under rotated coordinate systems with $\theta=45^\circ$, 60° , and 90° , and compare the results.

Q19 Use Matplotlib to reproduce Figure 10, plotting the same vector a in both the standard and the rotated coordinate systems.

Q21 Use Python to randomly generate an RGB color vector, compute the difference between the white vector and it, and verify the two chain tip-to-tail to recover the white vector (as in Figure 13).

Q23 Following Figure 12, write code to randomly generate 5 different non-orthogonal grids (bases) and plot the decomposition of $a=[3,4]^T$ under each.

Q18 Write a function `coordinates_in_basis(a, theta)` that returns a 's coordinates in the rotated square grid, and check it against `decompose()`.

Q20 In RGB space, use code to verify that white decomposes as "red + green + blue", and also as "yellow + blue", "magenta + green", and "cyan + red"; compare the lengths of each component.

Q22 Write a function `is_orthogonal_decomposition(v1, v2, sum_vec)` that checks whether two components are orthogonal and whether they sum to a target vector.

Q24 Combining this section's ideas, write a function `full_projection_report(a, b)` that reports the scalar projection, vector projection, orthogonal complement, and a 's coordinate along b all at once.

10 Prompts to Go Deeper (1/2)

1

Explain why scalar projection is just a number while vector projection is a vector — what question does each one answer?

2

Prove that any vector a can be uniquely decomposed into a vector projection onto a given direction b plus an orthogonal complement.

3

Use an everyday example to explain the "orthogonal complement" — such as the relationship between a shadow and the object casting it.

4

Why is the standard/orthonormal basis considered the "most natural" basis? What complications arise from using a non-orthogonal basis instead?

5

Explain the connection between scalar projection and cosine similarity — why can both measure "how close two directions are"?

10 Prompts to Go Deeper (2/2)

6

Design a small Python experiment verifying that the same vector has different coordinates under different "rotated square grids", even though the vector itself never changes.

7

Explain why there are infinitely many ways to decompose a vector, yet the standard/orthonormal basis decomposition is by far the most commonly used in practice.

8

If two direction vectors are not orthogonal (not a standard/orthonormal basis), does the scalar projection formula still hold? What adjustment is needed?

9

Using the RGB color space, discuss possible applications of "projection" and "orthogonal complement" in image processing or color separation.

10

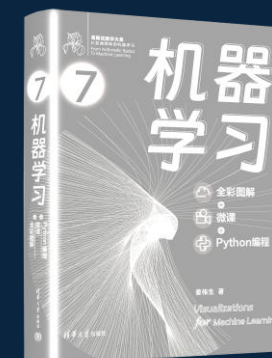
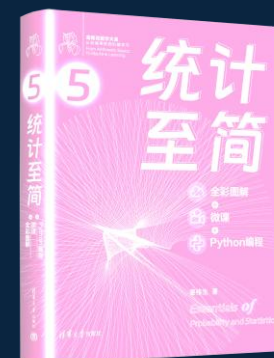
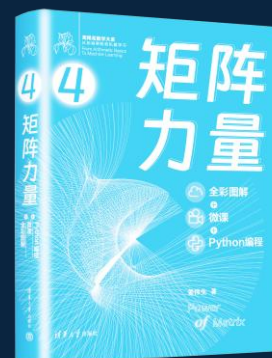
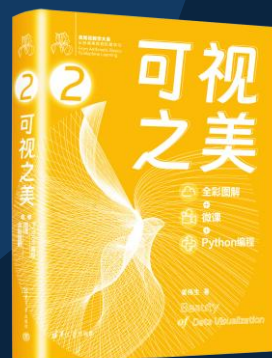
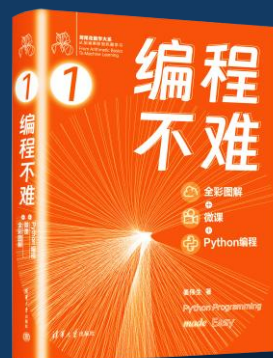
Describe a concrete application — such as robot pose estimation or signal decomposition — where orthogonal vector decomposition plays a practical role.

数学

数学基础

线性代数

概率统计



Python编程

数据可视化

工具

数据分析
图和网络

回归、降维
分类、聚类

实践

